

2. 算法

算法是对特定问题求解步骤的一种描述

- (1) 有穷性 有穷步骤 有穷时间
- (2) 确定性
- (3) 可行性
- (4) 输入 一个算法有零个或多个输入
- (5) 输出 一个算法有一个或多个输出

题1. 以下不属于算法特性的是 ()

- A. 可行性 B. 输入 C. 确定性 D. 健壮性

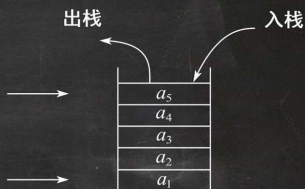
1. 栈的基本概念

栈： 只允许在一端进行插入或删除操作的线性表

栈顶 栈底

空栈： 不含任何元素的空表

特性： 先进后出



入: a_1, a_2, a_3, a_4, a_5

出: a_5, a_4, a_3, a_2, a_1

吐是栈

题1. 限定仅在表尾进行插入和删除操作的线性表称为 栈，它的修改是按 先进后出 的原则进行的

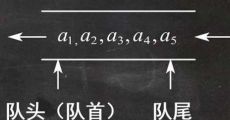
3. 队列的基本概念

队列： 队，只允许在表的一端进行插入，而在表的另一端进行删除

入队或进队

出队或离队

特性： 先进先出



入: a_1, a_2, a_3, a_4, a_5

出: a_1, a_2, a_3, a_4, a_5

拉屎是队列

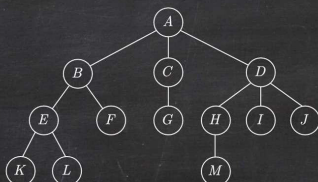
题1. 队列是一种操作受限的线性表，它与栈不同的是，队列中所有的插入操作均限制在表的一段进行，而所有的删除操作都限制在表的另一端进行，允许插入的一端称为 队尾，允许删除的一端称为 队首，队列具有 先进先出 的特点

FIFO

1. 树的基本概念

树: $n(n \geq 0)$ 个结点的有限集

空树



例1. 如果在数据结构中每个数据元素只能有一个直接前驱, 但可有多个直接后继, 则该结构是 ()

A. 栈

B. 队列

C. 树

D. 图

(1) 祖先、双亲、孩子 A, B, C 是 K 的祖先

(2) 结点的度: 该结点的孩子个数

树的度: 树中结点的最大度数

(3) 分支结点 (非终端结点): 度大于 0

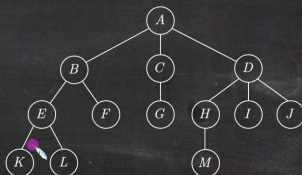
叶子结点 (终端结点): 度为 0

兄弟: 有相同双亲的结点

(4) 深度、高度

(5) 有序树、无序树

(6) 路径、路径长度



第 1 层

第 2 层

第 3 层

第 4 层

有序树

树的性质:

(1) 树中的结点数等于所有结点的度数加 1

(2) 度为 m 的树中第 i 层上至多有 m^{i-1} 个结点

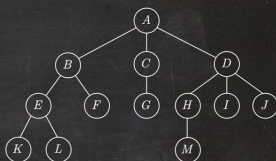
1	1
2	m
3	m^2

(3) 高度为 h 的 m 叉树至多有 $(m^h - 1) / (m - 1)$ 个结点

$$1 + m + m^2 + \dots + m^{h-1}$$

(4) 具有 n 个结点的 m 叉树的最小高度为 $\lceil \log_m (n(m-1) + 1) \rceil$

$$(m^h - 1) / (m - 1) = n$$



题2. 对一棵 n 个结点的树, 则该树中结点的度数之和为 ()

A. n

B. $n-1$

C. $n+1$

D. 不确定

树中的结点数等于所有结点的度数加 1

题3. 设一棵度为 3 的树中有 2 个度数为 1 的结点, 2 个度数为 2 的结点, 2 个度数为 3 的结点, 则该树中有 () 个度数为 0 的结点

A. 5

B. 6

C. 7

D. 8

设树中有 x 个度数为 0 的结点

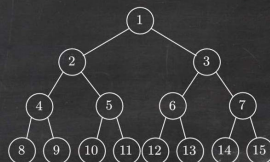
$$2 + 2 + 2 + x = 2 \times 1 + 2 \times 2 + 2 \times 3 + 1$$

2. 二叉树

几个特殊的二叉树:

(1) 满二叉树

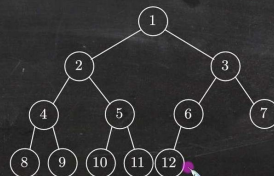
$$h \quad 2^h - 1$$



$$\lfloor i/2 \rfloor$$

$$2i \quad 2i+1$$

(2) 完全二叉树



$$i \leq \lfloor n/2 \rfloor$$

3. 二叉树的遍历

二叉树的遍历: 按某条搜索路径访问树中每个结点, 使得每个结点均被访问一次, 而且仅被访问一次

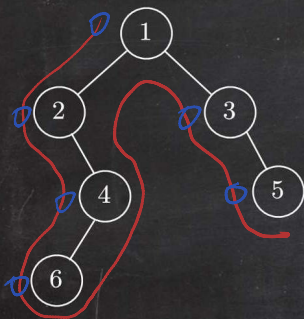
按照先遍历左子树再遍历右子树的原则, 常见的遍历次序有先序 (NLR)、中序 (LNR) 和后序 (LRN) 三种遍历算法

(1) 先序遍历 (前序遍历)

若二叉树为空, 则什么也不做; 否则,

- 1) 访问根结点
- 2) 先序遍历左子树
- 3) 先序遍历右子树

```
void PreOrder (BiTree T) {  
    if (T != NULL) {  
        visit(T);  
        PreOrder (T->lchild);  
        PreOrder (T->rchild);  
    }  
}
```

先序遍历 NLR

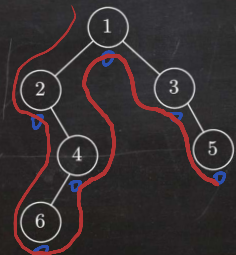
先序遍历所得到的结点序列为 1 2 4 6 3 5

(2) 中序遍历

若二叉树为空，则什么也不做，否则：

- 1) 中序遍历左子树
- 2) 访问根结点
- 3) 中序遍历右子树

```
void InOrder(BiTree T) {
    if (T != NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
```



中序遍历 LNR

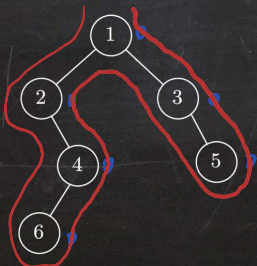
中序遍历所得到的结点序列为 2 6 4 1 3 5

(3) 后序遍历

若二叉树为空，则什么也不做，否则：

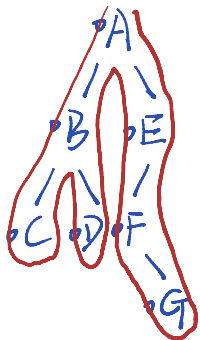
- 1) 后序遍历左子树
- 2) 后序遍历右子树
- 3) 访问根结点

```
void PostOrder(BiTree T) {
    if (T != NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

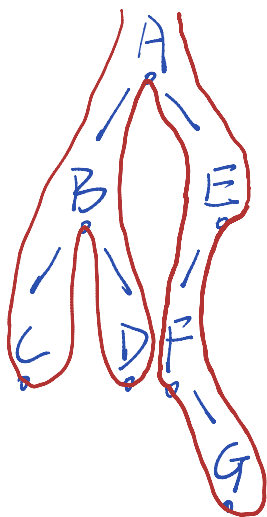


后序遍历 LRN

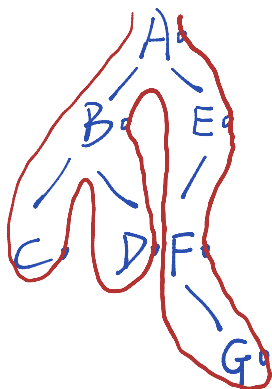
后序遍历所得到的结点序列为 6 4 2 5 3 1



前序遍历 ABCDEFG



中序遍历 CBDFGE



后序遍历 CDBGFE

4. 层序遍历

从树的第一层，即根结点开始访问

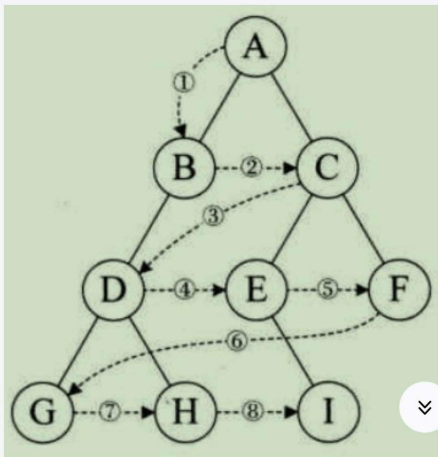
从上而下逐层遍历

在同一层中按从左到右的顺序对结点逐个访问

(总结：第一层 -> 第二层 (从左到右访问结点) -> ... -> 最后一层 (从左到右访问结点))

图形化介绍：

如下图所示，遍历的顺序为：ABCDEFGHI

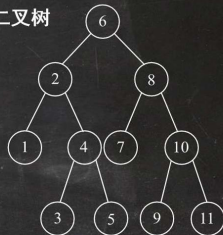


2. 二叉排序树

蜂考 bilibili

二叉排序树（二叉查找树）或者是一颗空树，或者是具有下列特性的二叉树

- (1) 若左子树非空，则左子树上所有结点的值均小于根结点的值
- (2) 若右子树非空，则右子树上所有结点的值均大于根结点的值
- (3) 左、右子树也分别是一棵二叉排序树



题1.若对一棵二叉排序树构成的序列采用中序遍历，可得到（ ）结果

A. 降序

B. 升序

C. 无序

D. 不确定



分析二叉排序树的定义，是左子树上的所有结点的值都小于根结点的值，右子树上的所有结点的值都大于根结点的值。例如图 7-7 所示二叉树满足任一结点的值均大于其左孩子的值，小于其右孩子的值，但不是二叉排序树。

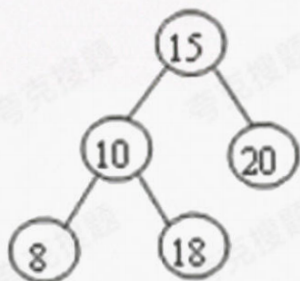


图 7-7 非二叉排序树

3. 哈夫曼树

权： 树中结点常被赋予一个代表某种意义的数值

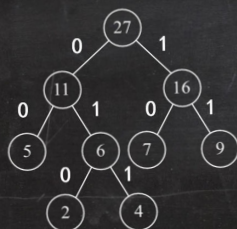
结点带权路径长度： 从树的根到任意结点的路径长度与该结点上权值的乘积

树的带权路径长度： 树中所有叶结点的带权路径长度之和，记为 $WPL = \sum_{i=1}^n w_i l_i$

哈夫曼树： 带权路径长度最小的二叉树 （最优二叉树）

题1. 有一电文使用五种字符 a, b, c, d, e ，其出现频率依次为 4, 7, 5, 2, 9

- (1) 试画出对应的哈夫曼树（要求左子树根结点的权小于等于右子树根结点的权）
- (2) 求出每个字符的哈夫曼编码
- (3) 译出编码序列 11000111000101011 的相应电文
- (4) 求带权路径长度



a 对应的哈夫曼编码为 011

b 对应的哈夫曼编码为 10 11000111000101011

c 对应的哈夫曼编码为 00 $e c a b c b b e$

d 对应的哈夫曼编码为 010

e 对应的哈夫曼编码为 11

$$WPL = 5 \times 2 + 2 \times 3 + 4 \times 3 + 7 \times 2 + 9 \times 2 = 60$$



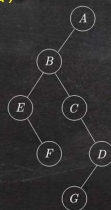
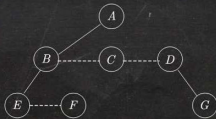
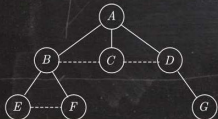
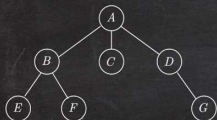
1. 树和森林

树的存储结构：双亲表示法、孩子表示法、孩子兄弟表示法

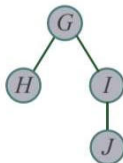
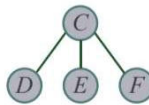
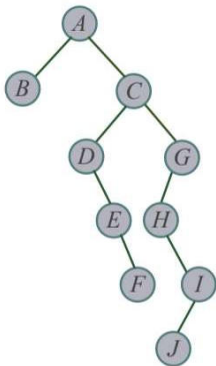
题1. 将树转换成二叉树

树转换成二叉树的画法：

- (1) 在兄弟结点之间加一条线
- (2) 对每个结点，只保留它与第一个孩子的连线，抹去与其他孩子的连线
- (3) 以树根为轴心，顺时针旋转 45°
(孩子在左，兄弟在右)



二叉树转换为树或森林

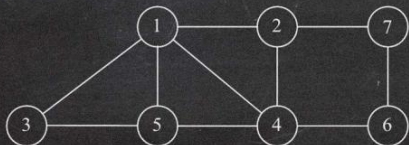


3. 图的遍历

图的遍历：从图中的某一顶点出发，按照某种搜索方法沿着图中的所有顶点访问一次且仅访问一次

深度优先遍历类似于树的前序遍历
广度优先遍历类似于树的层序遍历

题1. 如下图，从顶点 1 出发，按照广度优先规则遍历，可能得到的序列是 ()



1 3 5 4 2 6 7

答案不唯一

A. 1352467

1352476

B. 1423756

1423576

C. 1234576



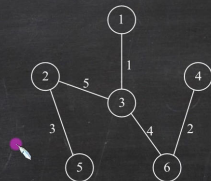
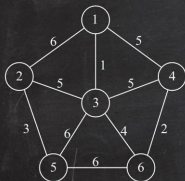
D. 1354672



1354267

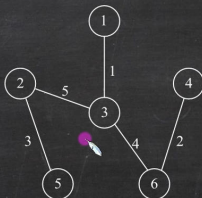
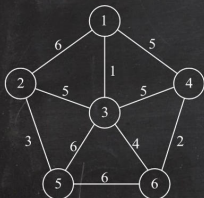
1. 最小生成树

最小生成树：带权连通无向图中，所有生成树中权值之和最小的生成树



Prim 算法：

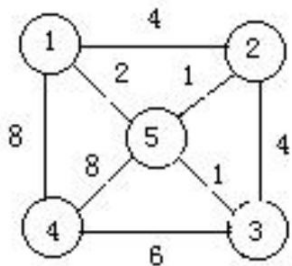
- ①任取一顶点，去掉所有边
- ②选择一个与当前顶点集合距离最近的顶点，并将该顶点和相应的边加入进来，同时不能形成回路
- ③重复②，直至图中所有顶点都并入

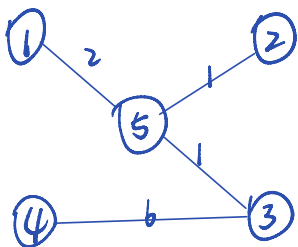


Kruskal 算法：

- ①去掉所有边
- ②选边（权最小，且不构成回路）
- ③重复②，直至图中所有顶点都并入

设无向图 G (如图所示)，给出该图的最小生成树上边的集合并计算最小生成树各边上的权值之和。

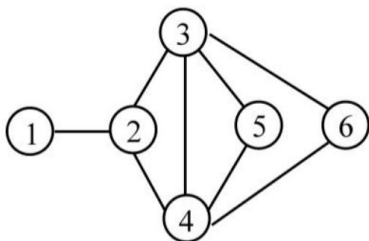




$$E = \{(1, 5), (5, 2), (5, 3), (3, 4)\}$$

$$W = 10$$

设有数据结构 (D, R) ，其中 $D = \{1, 2, 3, 4, 5, 6\}$ ， $R = \{(1, 2), (2, 3), (2, 4), (3, 4), (3, 5), (3, 6), (4, 5), (4, 6)\}$ 。试画出其逻辑结构图并指出属于何种结构。



图结构

图 1-3 逻辑结构图

例 9.5 带权图 G 如图 9.8 所示,求从 v_1 到其余各点的最短路径和距离.

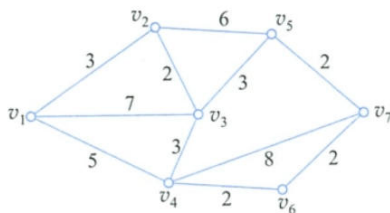


图 9.8

答案

解 用 Dijkstra 标号法求解,计算过程如表 9.1 所示.表中 * 表示永久标号, ** 表示这一步选中的永久标号,其余均是临时标号.

表 9.1 Dijkstra 标号法求解过程

步骤	v_1	v_2	v_3	v_4	v_5	v_6	v_7
1	$(v_1, 0)^{**}$	$(v_1, +\infty)$	$(v_1, +\infty)$	$(v_1, +\infty)$	$(v_1, +\infty)$	$(v_1, +\infty)$	$(v_1, +\infty)$
2	$(v_1, 0)^*$	$(v_1, 3)^{**}$	$(v_1, 7)$	$(v_1, 5)$	$(v_1, +\infty)$	$(v_1, +\infty)$	$(v_1, +\infty)$
3	$(v_1, 0)^*$	$(v_1, 3)^*$	$(v_2, 5)^{**}$	$(v_1, 5)$	$(v_2, 9)$	$(v_1, +\infty)$	$(v_1, +\infty)$
4	$(v_1, 0)^*$	$(v_1, 3)^*$	$(v_2, 5)^*$	$(v_1, 5)^{**}$	$(v_3, 8)$	$(v_1, +\infty)$	$(v_1, +\infty)$
5	$(v_1, 0)^*$	$(v_1, 3)^*$	$(v_2, 5)^*$	$(v_1, 5)^*$	$(v_3, 8)$	$(v_4, 7)^{**}$	$(v_4, 13)$
6	$(v_1, 0)^*$	$(v_1, 3)^*$	$(v_2, 5)^*$	$(v_1, 5)^*$	$(v_3, 8)^{**}$	$(v_4, 7)^*$	$(v_6, 9)$
7	$(v_1, 0)^*$	$(v_1, 3)^*$	$(v_2, 5)^*$	$(v_1, 5)^*$	$(v_3, 8)^*$	$(v_4, 7)^*$	$(v_6, 9)^{**}$

根据表 9.1 的最后一行,从 v_1 到其余各点的最短路径和距离如下.

$$v_1 v_2, \quad d(v_1, v_2) = 3$$

$$v_1 v_2 v_3, \quad d(v_1, v_3) = 5$$

$$v_1 v_4, \quad d(v_1, v_4) = 5$$

$$v_1 v_2 v_3 v_5, \quad d(v_1, v_5) = 8$$

$$v_1 v_4 v_6, \quad d(v_1, v_6) = 7$$

$$v_1 v_4 v_6 v_7, \quad d(v_1, v_7) = 9$$

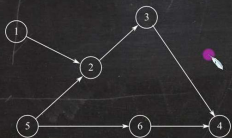
3. 拓扑排序 有向无环图

AOV 网: 顶点表示活动 $\langle V_i, V_j \rangle$

拓扑排序:

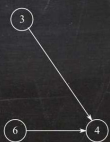
- ① 每个顶点出现且只出现一次
- ② 若存在一条从顶点 A 到顶点 B 的路径, 则在排序中顶点 B 出现在顶点 A 的后面

题1. 下图的一个拓序排序序列为 _____



拓扑排序的算法的步骤:

- ① 从 AOV 网中选择一个没有前驱的顶点并输出
- ② 从网中删除该顶点和所有以它为起点的有向边
- ③ 重复①和②直到当前的 AOV 网为空或当前网中不存在无前驱的顶点为止

出现在顶点 A 的后面题1. 下图的一个拓序排序序列为 152364

1523

拓扑排序的算法的步骤:

- ① 从 AOV 网中选择一个没有前驱的顶点并输出
- ② 从网中删除该顶点和所有以它为起点的有向边
- ③ 重复①和②直到当前的 AOV 网为空或当前网中不存在无前驱的顶点为止

4. 关键路径

AOE网：以顶点表示事件，以有向边表示活动，以边上的权值表示完成该活动的开销

关键路径：从开始顶点到结束顶点的所有路径中，具有最大路径长度的路径

关键活动：关键路径上的活动

题1. 关键路径是事件网络中 ()

A. 从源点到汇点的最短路径

B. 从源点到汇点的最长路径

C. 最长的回路

D. 最短的回路

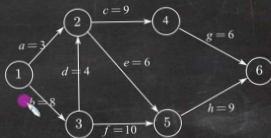
(1) 事件 v_k 的最早发生时间 $ve(k)$

指从源点 v_1 到顶点 v_k 的最长路径长度

$$ve(\text{源点}) = 0$$

$$ve(k) = \max\{ve(j) + \text{Weight}(v_j, v_k)\}$$

从前往后



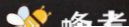
(2) 事件 v_k 的最迟发生时间 $vl(k)$

指在不推迟整个工程完成的前提下，即保证它的后续事件 v_j 在其最迟发生时间 $vl(j)$ 能够发生时，该事件最迟必须发生的时间

$$vl(\text{汇点}) = ve(\text{汇点})$$

$$vl(k) = \min\{vl(j) - \text{Weight}(v_k, v_j)\}$$

从后往前



(3) 活动 a_i 的最早开始时间 $e(i)$

指该活动弧的起点所表示的事件的最早发生时间

$$\langle v_k, v_j \rangle$$

a_i

$$e(i) = ve(k)$$

(4) 活动 a_i 的最迟开始时间 $l(i)$

指该活动弧的终点所表示事件的最迟发生时间与该活动所需时间之差

$$\langle v_k, v_j \rangle$$

a_i

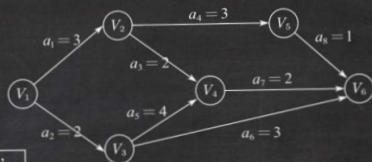
$$l(i) = vl(j) - \text{Weight}(v_k, v_j)$$

(5) 一个活动 a_i 的最迟开始时间 $l(i)$ 和其最早开始时间 $e(i)$ 的差额 $d(i) = l(i) - e(i)$

$l(i) - e(i) = 0$ 的活动 a_i 是关键活动

4. 关键路径

题2.求下图的关键路径



求关键路径的算法步骤如下:

①令 ve (源点) = 0, 求最早发生时间 $ve()$ ②令 vl (汇点) = ve (汇点), 求最迟发生时间 $vl()$ ③根据 $ve()$ 值求所有弧的最早开始时间 $e()$ ④根据 $vl()$ 值求所有弧的最迟开始时间 $l()$ ⑤求 AOE 网中所有活动的差额 $d()$, 找出所有 $d() = 0$ 的活动构成关键路径

	v_1	v_2	v_3	v_4	v_5	v_6
$ve(i)$	0	3	2	6	6	8
$vl(i)$	0	4	2	6	7	8

	a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8
$e(i)$	0	0	3	3	2	2	6	6
$l(i)$	1	0	4	4	2	5	6	7
$d(i)$	1	0	1	1	0	3	0	1

关键路径 (v_1, v_3, v_4, v_6)

1. 查找

(1) 查找：在数据集中，寻找满足某种条件的数据元素的过程

(2) 查找表（查找结构）：用于查找的数据集合

(3) 关键字：数据元素中某个数据项的值，用它可以标识一个数据元素

主关键字：关键字可以唯一地标识一个记录

(4) 平均查找长度：所有查找过程中进行关键字的比较次数的平均值 $ASL = \sum_{i=1}^n P_i C_i$

2. 顺序查找

基本思想：从线性表的一端开始，逐个检查关键字是否满足给定的条件。若查找到某个元素的关键字满足条件，则查找成功，返回该元素在线性表中的位置。若查找找到表的另一端，仍未找到符合给定条件的元素，则返回查找失败的信息



```
typedef struct {
    ElemType *elem;
    int TableLen;
} SSTable;

int Search_Seq(SSTable ST, ElemType key) {
    ST.elem[0] = key;
    for (i = ST.TableLen; ST.elem[i] != key; ++i);
    return i;
}
```

注意：对线性链表只能进行顺序查找

题1. 采用顺序查找方法查找长度为 n 的顺序表时，查找成功的平均查找长度为（ ）

A. n

B. $n/2$

C. $(n+1)/2$

D. $(n-1)/2$

1

n

2

$n-1$

3

$n-2$

i

$n-i+1$

n

1

$$ASL = \sum_{i=1}^n P_i C_i$$

查找成功时，顺序查找的平均查找长度为 $ASL_{成功} = \sum_{i=1}^n \frac{1}{n} (n-i+1) = \frac{n+1}{2}$

顺序查找不成功的平均查找长度为 $ASL_{不成功} = n+1$

3. 折半查找

折半查找（二分查找），仅适用于有序的顺序表

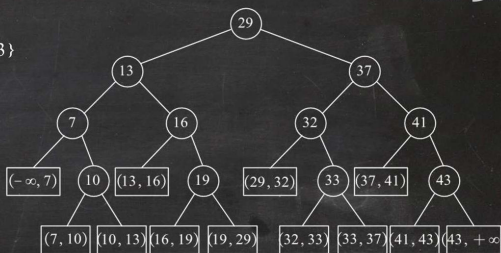
$\{7, 10, 13, 16, 19, 29, 32, 33, 37, 41, 43\}$ $key = 11$

low	0	1	2	3	4	5	6	7	8	9	10
$high$	7	10	13	16	19	29	32	33	37	41	43
mid	$low \uparrow high$										
	$\uparrow mid$										
	$mid = \lfloor (low + high) / 2 \rfloor$										
	$low \leq high$										
	$key < A[mid] \quad high = mid - 1$										
	$key > A[mid] \quad low = mid + 1$										

3. 折半查找

蜂考 bilibili

$\{7, 10, 13, 16, 19, 29, 32, 33, 37, 41, 43\}$



查找成功的平均查找长度是 $ASL = \frac{1}{n} \sum_{i=1}^n l_i = \frac{1}{n} (1 \times 1 + 2 \times 2 + \dots + h \times 2^{h-1})$

$$h = \lceil \log_2(n+1) \rceil = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

折半查找的时间复杂度为 $O(\log_2 n)$

蜂考

题1. 二分查找要求结点 ()

A. 有序、顺序存储

B. 有序、链式存储

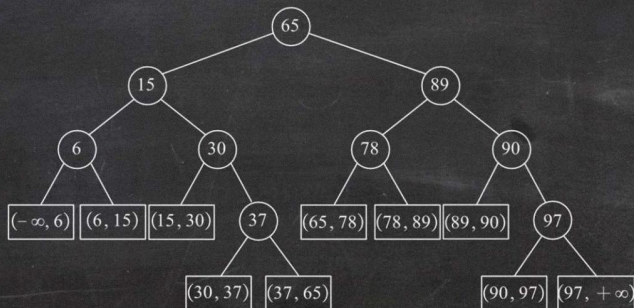
C. 无序、顺序存储

D. 无序、链式存储

折半查找（二分查找），仅适用于有序的顺序表

题2. 采用对分查找序列数据 (6, 15, 30, 37, 65, 78, 89, 90, 97) , 若查找元素 89 时比较

次数为 2



计算平均查找长度：

根据平均查找长度的计算公式 $ASL_{成功} =$

$\sum_{i=1}^n p_i c_i$ (其中 p_i 是查找第 i 个元素的概率,
 c_i 是查找第 i 个元素的比较次数) , 在等概率情

况下 $p_i = \frac{1}{n}$ 。

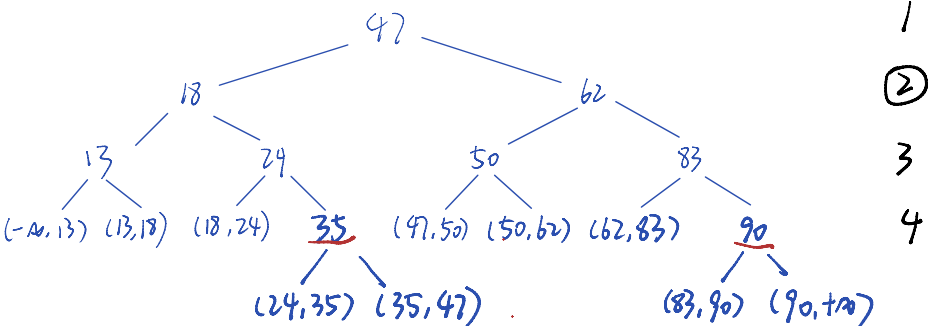
根据平均查找长度的计算公式 $ASL_{失败} =$

$\sum_{i=1}^{n+1} p_i c_i$ (其中 p_i 是查找第 i 个失败位置的概
率, c_i 是查找第 i 个失败位置的比较次数) , 在

等概率情况下 $p_i = \frac{1}{n+1}$ 。

五、计算题：（每题 9 分，共 27 分） 长度 $n=9$

1. 一组有序的记录关键字序列为 (13, 18, 24, 35, 47, 50, 62, 83, 90)，查找方法用二分查找，要求计算出查找关键字 62 时的比较次数并计算出查找成功时的平均查找长度。



1. 查找关键字 62 时的比较次数

二分查找过程（序列：13, 18, 24, 35, 47, 50, 62, 83, 90，长度 $n=9$ ）

- 第 1 次比较：中间元素为第 5 个元素（索引 4，0 开始），即 47。62 > 47，确定在右半部分（50, 62, 83, 90）。
- 第 2 次比较：右半部分中间元素为第 7 个元素（索引 6），即 62。62 = 62，查找成功。

结论：查找关键字 62 时的比较次数为 2 次。

2. 查找成功时的平均查找长度（ASL）

平均查找长度公式： $ASL = \frac{\sum_{i=1}^n \text{比较次数}_i}{n}$

$$\frac{1 + 2 \times 2 + 3 \times 4 + 4 \times 2}{9} = \frac{25}{9}$$

长度为 12 的有序表采用顺序存储结构，采用折半查找技术，在等概率情况下，查找成功时的平均查找长度是（），查找失败时的平均查找长度是（）**AB**

A、37/12

B、62/13

C、39/12

D、49/13

$$ASL_{\text{成功}} = \frac{1 + 2 \times 2 + 3 \times 4 + 4 \times 5}{12} = \frac{1 + 4 + 12 + 20}{12} = \frac{37}{12}$$

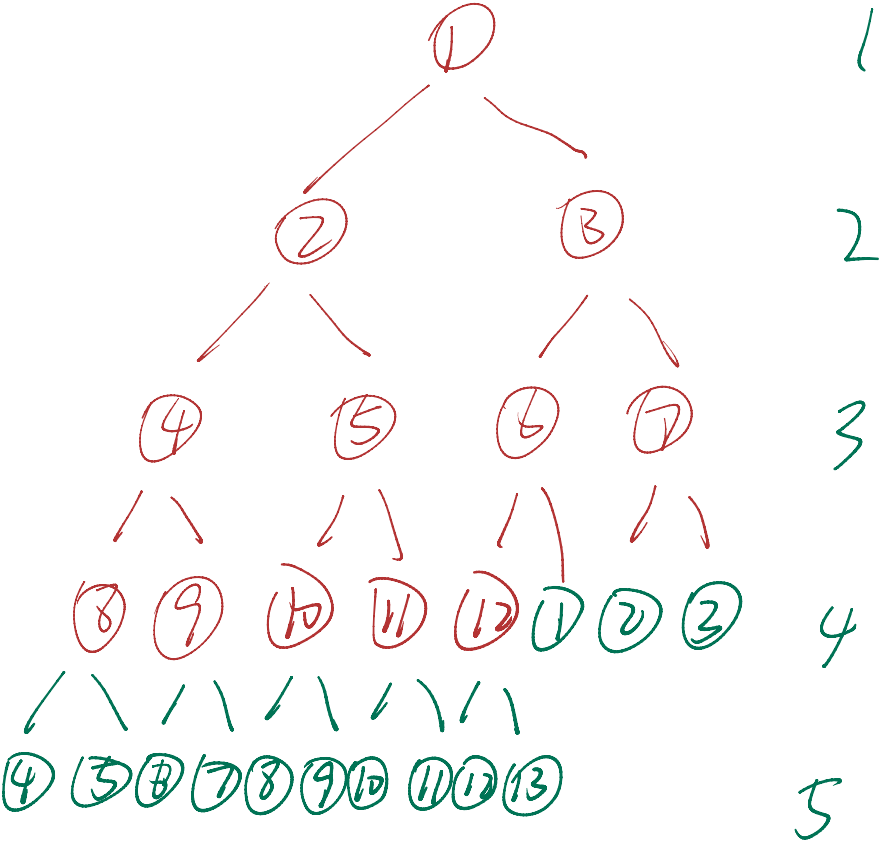
$$ASL_{\text{失败}} = \frac{4 \times 3 + 5 \times 10}{13} = \frac{12 + 50}{13} = \frac{62}{13}$$

2. 计算查找成功时的平均查找长度 $ASL_{成功}$

• 确定比较次数：

对于长度为 $n = 12$ 的有序表，折半查找的过程可以用一棵完全二叉树来表示。在折半查找中，每次比较都将查找区间缩小一半。查找成功时，比较次数与元素在有序表中的位置有关。

- 第 1 层（根节点）：比较 1 次，有 1 个元素。
- 第 2 层：比较 2 次，有 2 个元素。
- 第 3 层：比较 3 次，有 4 个元素。
- 第 4 层：比较 4 次，有 5 个元素（因为 $1 + 2 + 4 = 7$ ，还剩 $12 - 7 = 5$ 个元素）。



4. 散列表

散列函数: $Hash(key) = Addr$

同义词: 散列函数可能会把两个或两个以上的不同关键字映射到同一地址

$3 \% 4 = 3$ $7 \% 4 = 3$

(1) 直接定址法: $H(key) = key$ $H(key) = a \times key + b$

(2) 除留余数法: m 取一个不大于但最接近或等于 m 的质数 p

$H(key) = key \% p$

(3) 数字分析法

(4) 平方取中法

解决冲突的方法: $3 \% 4 = 3$ $7 \% 4 = 3$

(1) 开放定址法: 可存放新表项的空闲地址既向它的同义词表项开放, 又向它的非同义词表项开放。其数学递推公式为 $H_i = (H(key) + d_i) \% m$

①线性探测法 $d_i = 0, 1, 2, \dots, m - 1$

②平方探测法 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ $k \leq m/2$

③再散列法 $d_i = Hash_2(key)$

题1. 关键字序列为 {19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79} , 按散列函数 $H(key) = key \% 13$ 和线性探测处理冲突构造散列表, 其中散列表长度为 16

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
关键字		14	01	68	27	55	19	20	84	79	23	11	10			
比较次数		1	2	1	4	3	1	1	3	9	1	1	3			

$H(19) = 19 \% 13 = 6$ $H(84) = 84 \% 13 = 6$

$H(14) = 14 \% 13 = 1$ $H(27) = 27 \% 13 = 1$

$H(23) = 23 \% 13 = 10$ $H(55) = 55 \% 13 = 3$

$H(01) = 01 \% 13 = 1$ $H(11) = 11 \% 13 = 11$

$H(68) = 68 \% 13 = 3$ $H(10) = 10 \% 13 = 10$

$H(20) = 20 \% 13 = 7$ $H(79) = 79 \% 13 = 1$

$H_i = (H(key) + d_i) \% m$

①线性探测法

$d_i = 0, 1, 2, \dots, m - 1$



查找成功的平均查找长度 $ASL_{成功} = \text{查找次数} / \text{元素个数}$

$$(1 \times 6 + 2 + 3 \times 3 + 4 + 9) / 12 = 2.5$$

查找不成功的平均查找长度为 $ASL_{不成功} = \text{查找次数} / \text{散列后的地址个数}$

$$(1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2) / 13 = 7$$

拉链法 (链地址法)

(2) 拉链法: 为了避免非同义词发生冲突, 把所有的同义词存储在一个线性链表中

题2. 关键字序列为 {19, 14, 23, 01, 68, 20, 84, 27, 55, 11, 10, 79}, 散列函数为

$H(\text{key}) = \text{key} \% 13$, 用拉链法解决冲突

$$H(19) = 19 \% 13 = 6$$

$$H(14) = 14 \% 13 = 1$$

$$H(23) = 23 \% 13 = 10$$

$$H(01) = 01 \% 13 = 1$$

$$H(68) = 68 \% 13 = 3$$

$$H(20) = 20 \% 13 = 7$$

$$H(84) = 84 \% 13 = 6$$

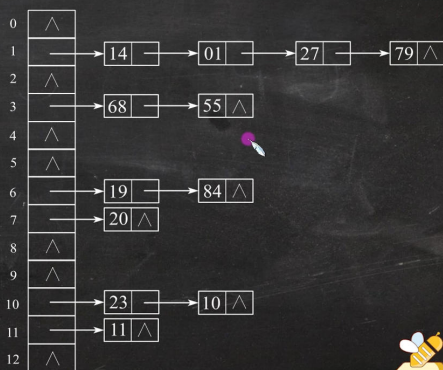
$$H(27) = 27 \% 13 = 1$$

$$H(55) = 55 \% 13 = 3$$

$$H(11) = 11 \% 13 = 11$$

$$H(10) = 10 \% 13 = 10$$

$$H(79) = 79 \% 13 = 1$$



查找成功的平均查找长度为

$$(1 \times 6 + 2 \times 4 + 3 + 4) / 12 = 1.75$$

查找不成功的平均查找长度为

$$(1 \times 7 + 2 \times 2 + 3 \times 3 + 5) / 13 = 25 / 13$$

3. 散列表的长度为 8，散列函数 $H(k) = k \bmod 7$ ，初始记录关键字序列为 (25, 31, 8, 27, 13, 68)，要求分别计算出用线性探测法和链地址法作为解决冲突方法的平均查找长度。

线性探测法：

$$H(25) = 25 \bmod 7 = 4$$

$$H(31) = 31 \bmod 7 = 3$$

$$H(8) = 8 \bmod 7 = 1$$

$$H(27) = 27 \bmod 7 = 6$$

$$H(13) = 13 \bmod 7 = 6$$

$$H(68) = 68 \bmod 7 = 5$$

	0	1	2	3	4	5	6	7
关键字		8		31	25	68	27	13
比较次数		1		1	1	1	1	2

$$ASL_1 = \frac{5+2}{6} = \frac{7}{6}$$

链地址法：

$$ASL_2 = \frac{5+2}{6} = \frac{7}{6}$$

0	^	
1		8 ^
2		^
3		31 ^
4		25 ^
5		68 ^
6		27 13 ^

已知一个连通图如图6-8所示，试给出图的邻接矩阵和邻接表存储示意图，若从顶点v1出发对该图进行遍历，分别给出一个按深度优先遍历和广度优先遍历的顶点序列。

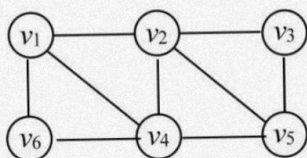


图 6-8 第 (4) 题图

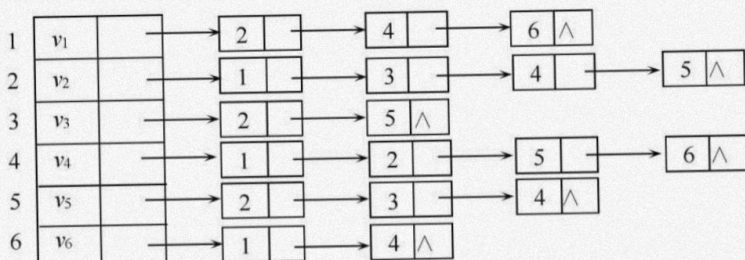
邻接矩阵表示如下：

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

深度优先遍历序列为：v1 v2 v3 v5 v4 v6

广度优先遍历序列为：v1 v2 v4 v6 v3 v5

邻接表表示如下：



2. 插入排序

基本思想是每次将一个待排序的记录按其关键字大小插入到前面已排好序的子序列中，直到全部记录插入完成

(1) 直接插入排序 49, 38, 65, 97, 76, 13, 27, 49

$i=2$	38	49	65	97	76	13	27	49	第一趟	<pre>void InsertSort(ElemType A[], int n) { int i, j; for (i=2; i<=n; i++) if (A[i]<A[i-1]) { A[0]=A[i]; for (j=i-1; A[0]<A[j]; --j) A[j+1]=A[j]; A[j+1]=A[0]; } }</pre>
$i=3$	38	49	65	97	76	13	27	49		
$i=4$	38	49	65	97	76	13	27	49		
$i=5$	38	49	65	76	97	13	27	49		
$i=6$	13	38	49	65	76	97	27	49		
$i=7$	13	27	38	49	65	76	97	49		
$i=8$	13	27	38	49	65	76	97			
$i=8$	13	27	38	49	65	76	97			

空间效率: $O(1)$

时间效率: $O(n^2)$

稳定性: 稳定

3. 交换排序

(2) 快速排序

基本思想是：在待排序表 $L[1 \cdots n]$ 中任取一个元素 *pivot* 作为枢轴（通常取首元素），通过一趟快速排序将待排序表划分为独立的两部分 $L[1 \cdots k-1]$ 和 $L[k+1 \cdots n]$ ，*pivot* 放在了最终位置 $L[k]$ 中 左侧记录均小于等于轴值，右侧均大于等于轴值

题1. 对关键字序列 {49, 38, 65, 97, 76, 13, 27, 49'} 进行快速排序

49	38	65	97	76	13	27	49'	$pivot = 49$
27	38	13	49	76	97	65	49'	
								$i \parallel j$
13	27	38						$pivot = 27$
								$i \parallel j$
13	27	38	49	49'	65	76	97	

空间效率: $O(\log_2 n)$

时间效率: $O(n \log_2 n)$

稳定性: 不稳定

23 13 35 6 19 50 28

23 13 35 6 19 50 28

i ↑ j ↑

19 13 35 6 23 50 28

i j

19 13 35 6 23 50 28

i j

19 13 23 6 35 50 28

i j

19 13 6 23 35 50 28

i j

图 8-7 一次划分的过程示例

23 13 35 6 19 50 28

19 13 6 **23** 35 50 28

6 13 19 23 28 35 50

6 13 19 23 28 35 50

6 13 19 23 28 35 50

图 8-8 快速排序的执行过程

快速排序是所有内部排序算法中平均性能最优的排序算法

快速排序并不适用于原本有序或基本有序的记录序列进行排序

1. 选择排序

(1) 简单选择排序（直接选择排序）

基本思想：假设排序表为 $L[1 \cdots n]$ ，第 i 趟排序即从 $L[i \cdots n]$ 中选择关键字最小的元素与 $L[i]$ 交换

题1. 给定关键字序列 {87, 45, 78, 32, 17, 65, 53, 09}，用简单选择排序算法进行排序

	87	45	78	32	17	65	53	09
第一趟排序	09	45	78	32	17	65	53	87
第二趟排序	09	17	78	32	45	65	53	87
第三趟排序	09	17	32	78	45	65	53	87
第四趟排序	09	17	32	45	78	65	53	87
第五趟排序	09	17	32	45	53	65	78	87
第六趟排序	09	17	32	45	53	65	78	87
第七趟排序	09	17	32	45	53	65	78	87

```
void SelectSort(ElemType A[], int n) {  
    for (i=0; i<n-1; i++) {  
        min=i;  
        for (j=i+1; j<n; j++)  
            if (A[j]<A[min])  
                min=j;  
        if (min!=i)  
            swap (A[i], A[min]);  
    }  
}
```

空间效率： $O(1)$

时间效率： $O(n^2)$

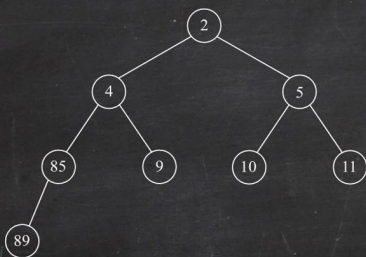
稳定性： 不稳定

{2, 2', 1}

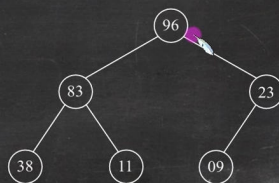
{1, 2', 2}



(2) 堆排序



2, 4, 5, 85, 9, 10, 11, 89

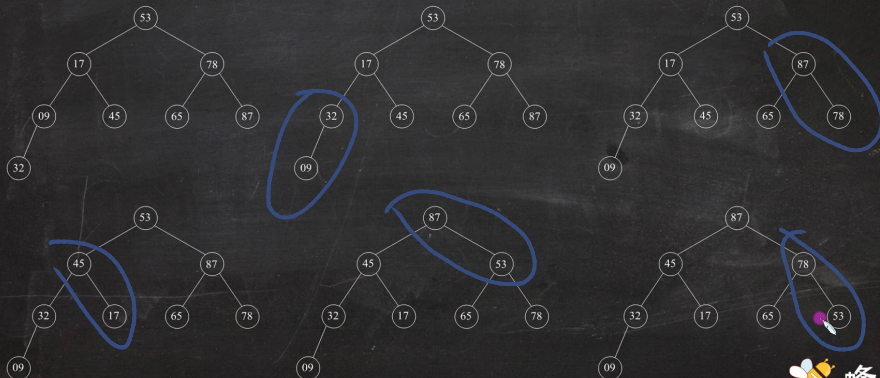


96, 83, 23, 38, 11, 09

53, 17, 78, 09, 45, 65, 87, 32

构造大根堆

$\lfloor n/2 \rfloor$



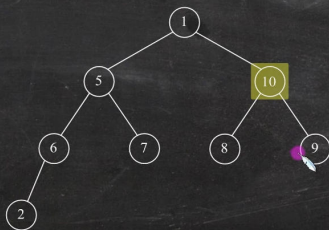
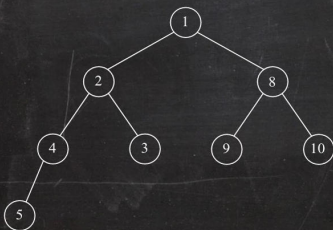
题1.下面的序列中，（ ）是堆

A. 1, 2, 8, 4, 3, 9, 10, 5

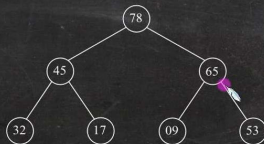
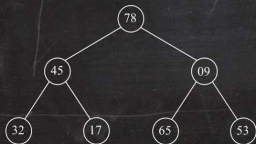
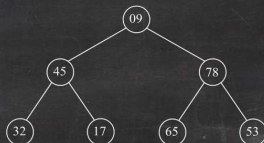
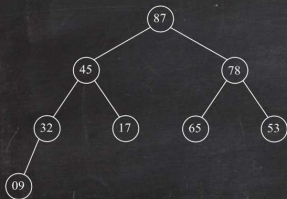
B. 1, 5, 10, 6, 7, 8, 9, 2

C. 9, 8, 7, 6, 4, 8, 2, 1

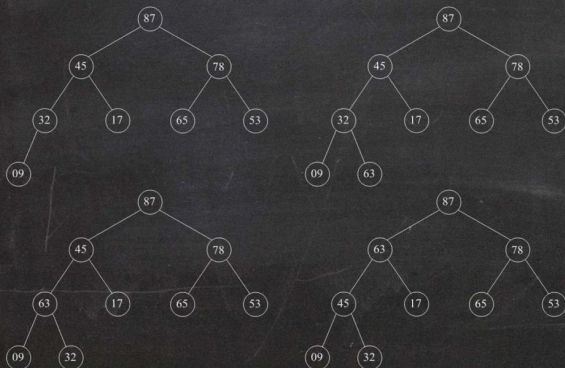
D. 9, 8, 7, 6, 5, 4, 3, 7



输出栈顶元素后，将堆的最后一个元素与栈顶元素交换，此时堆的性质被破坏



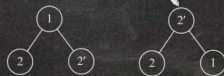
同时，堆也支持插入操作。对堆进行插入操作时，先将新结点放在堆的末端，再对这个新结点向上执行调整操作



空间效率: $O(1)$

时间效率: $O(n \log_2 n)$

稳定性: 不稳定



题2. 设有5000个待排序的记录关键字，如果需要用最快的方法选出其中最小的10个记录关键字，则用下列（ ）方法可以达到目的

A. 快速排序

B. 堆排序

C. 冒泡排序

D. 希尔排序

设有一个空栈，栈顶指针为1000H，现有输入序列为1、2、3、4、5，经过push, push, pop, push, pop, push, push后，输出序列是 1，栈顶指针为 2

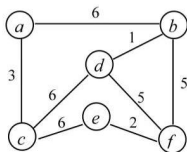
本题涉及栈的数据结构操作，包括进栈（push）和出栈（pop）。栈的特点是后进先出（LIFO），即最后压入栈的元素最先弹出。我们需要按照给定的操作顺序，模拟栈的变化过程，并确定最终的栈顶指针位置。

1. 初始状态：栈顶指针为1000H，栈为空。
2. 操作push(1)：栈顶指针向下移动一个位置，栈顶指针变为1001H，栈中元素为1。
3. 操作push(2)：栈顶指针向下移动一个位置，栈顶指针变为1002H，栈中元素为1, 2。
4. 操作pop()：栈顶元素2出栈，栈顶指针向上移动一个位置，栈顶指针变为1001H，栈中元素为1。
5. 操作push(3)：栈顶指针向下移动一个位置，栈顶指针变为1002H，栈中元素为1, 3。
6. 操作pop()：栈顶元素3出栈，栈顶指针向上移动一个位置，栈顶指针变为1001H，栈中元素为1。
7. 操作push(4)：栈顶指针向下移动一个位置，栈顶指针变为1002H，栈中元素为1, 4。
8. 操作push(5)：栈顶指针向下移动一个位置，栈顶指针变为1003H，栈中元素为1, 4, 5。

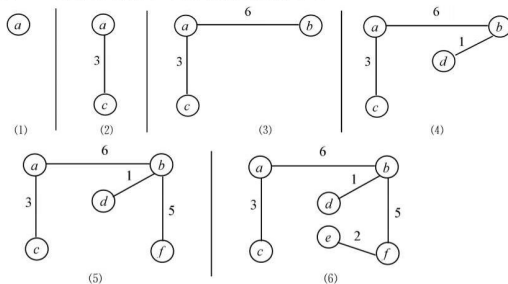
最终，栈顶指针位置为1003H，栈中的元素为1, 4, 5。

故答案为: 23; 1003H

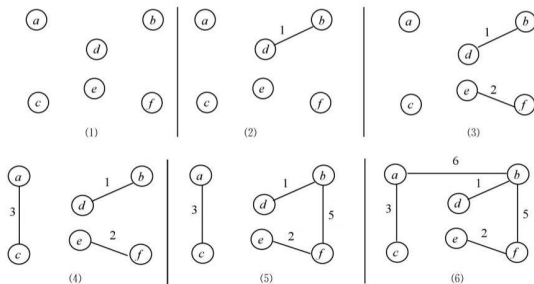
图6-9所示是一个无向带权图，请分别按Prim算法和Kruskal算法求最小生成树。



按Prim算法求最小生成树的过程如下：



按Kruskal算法求最小生成树的过程如下：



对如图6-5所示的无向连通网图从顶点d开始用Prim算法构造最小生成树，在构造过程中加入最小生成树的前4条边依次是（ ）。

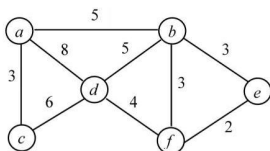


图 6-5 一个无向连通网

先将顶点d涂黑，然后选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(d, f)4；然后将顶点f涂黑，选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(f, e)2；然后将顶点e涂黑，再选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(f, b)3或(e, b)3；然后将顶点b涂黑，再选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(b, a)5。

对于如图6-11所示的有向网图，求出各活动的最早开始时间和最晚开始时间，并写出关键活动和关键路径。

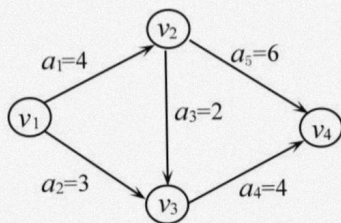


图 6-11 一个有向网图

事件	v_1	v_2	v_3	v_4
最早开始时间	0	4	6	10
最迟开始时间	0	4	6	10

活动	a_1	a_2	a_3	a_4	a_5
最早开始时间	0	0	4	6	4
最迟开始时间	0	3	4	6	4

关键活动是 a_1 、 a_3 、 a_4 和 a_5 ，它们组成两条关键路径，如图6-12所示。

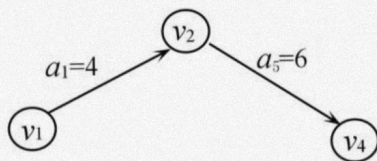
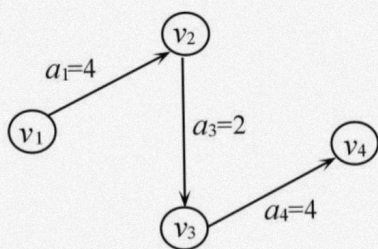


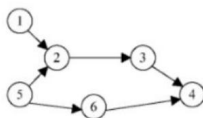
图 6-12 两条关键路径

关键路径: (v_1, v_2, v_3, v_4) , (v_1, v_2, v_4)

已知如下长度为12的表 (Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec)，试按表中元素的顺序依次插入一棵初始为空的二叉排序树,画出插入完成之后的二叉排序树



试列出下图中全部可能的拓扑有序序列



152364
152634
156234
512364
512634
516234
561234

判别下列序列是否为堆，如不是，把它调整为堆，用图表示建堆的过程。

① (1, 5, 7, 25, 21, 8, 8, 42)

② (3, 9, 5, 8, 4, 17, 21, 6)

序列①是堆，序列②不是堆，调整为堆（假设为大根堆）的过程如图8-6所示。

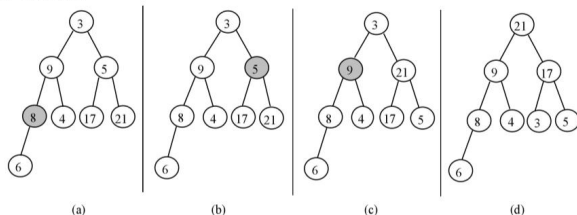


图 8-6 堆调整的过程

已知数据序列为(12, 5, 9, 20, 6, 31, 24), 对该数据序列进行排序, 写出插入排序、起泡排序、快速排序、简单选择排序、堆排序以及二路归并排序每趟的结果。

插入排序：

初始键值序列 [12] 5 9 20 6 31 24

第一趟结果 [5 12] 9 20 6 31 24

第二趟结果 [5 9 12] 20 6 31 24

第三趟结果 [5 9 12 20] 6 31 24

第四趟结果 [5 6 9 12 20] 31 24

第五趟结果 [5 6 9 12 20 31] 24

第六趟结果 [5 6 9 12 20 24 31]

简单选择排序：

初始键值序列 [12 5 9 20 6 31 24]

第一趟结果 5 [12 9 20 6 31 24]

第二趟结果 5 6 [9 20 12 31 24]

第三趟结果 5 6 9 [20 12 31 24]

第四趟结果 5 6 9 12 [20 31 24]

第五趟结果 5 6 9 12 20 [31 24]

第六趟结果 5 6 9 12 20 24 31

快速排序：

初始键值序列 12 5 9 20 6 31 24

第一趟结果 [6 5 9] 12 [20 31 24]

第二趟结果 [5] 6 [9] 12 20 [31 24]

第三趟结果 5 6 9 12 20 [24] 31

第四趟结果 5 6 9 12 20 24 31

起泡排序：

初始键值序列 [12 5 9 20 6 31 24]

第一趟结果 [5 9 12 6 20 24] 31

第二趟结果 [5 9 6] 12 20 24 31

第三趟结果 [5 6] 9 12 20 24 31

第四趟结果 5 6 9 12 20 24 31

堆排序：

初始键值序列 [12 5 9 20 6 31 24]

初始建堆结果 [31 20 24 5 6 9 12]

第一趟结果 [24 20 12 5 6 9] 31

第二趟结果 [20 9 12 5 6] 24 31

第三趟结果 [12 9 6 5] 20 24 31

第四趟结果 [9 5 6] 12 20 24 31

第五趟结果 [6 5] 9 12 20 24 31

第六趟结果 5 6 9 12 20 24 31

二路归并排序：

初始键值序列 12 5 9 20 6 31 24

第一趟结果 [5 12] [9 20] [6 31] [24]

第二趟结果 [5 9 12 20] [6 24 31]

第三趟结果 [5 6 9 12 20 24 31]